

Resumen clase 18/08 - Procesos y APIs

Procesos

¿Qué es un proceso?

Componentes principales de un proceso:

¿Qué puede hacer un proceso?

Terminar

Lanzar un proceso hijo

Ejecutar en la CPU

 Componentes que intervienen

 Ejecución y cambios de contexto

 Quantum y planificación

Hacer un system call.

Syscall

Wrapper functions

Realizar entrada/salida a los dispositivos (E/S)

Busy waiting

Polling

Interrupciones

E/S bloqueante vs. no bloqueante

Estados de un proceso

Árbol de procesos

Señales y Handlers

Procesos

¿Qué es un proceso?

Un **programa** es una secuencia de pasos escrita en algún lenguaje. Ese programa eventualmente se compila en código objeto, lo que también es un programa escrito en lenguaje de máquina.

Cuando ese programa se pone a ejecutar, lo que tenemos es un **proceso**.

Se diferencia de un programa porque:

- El programa = archivo estático (código en disco).
- El proceso = entidad dinámica (tiene estado, recursos, contexto).

Componentes principales de un proceso:

- **Código ejecutable** (instrucciones del programa).
 - **Datos asociados** (variables globales, memoria dinámica).
 - **Pila (stack)** con parámetros y llamadas de funciones.
 - **Contador de programa (PC)** que indica la próxima instrucción.
 - **Registros de CPU y estado.**
 - **Recursos asignados** (archivos abiertos, sockets, memoria).
 - **Se le asigna un identificador numérico único, el pid o process id.**
-

¿Qué puede hacer un proceso?

Terminar

- `exit()`
 - Termina la ejecución del proceso.
 - Devuelve valor de salida al padre.

Lanzar un proceso hijo

- El sistema operativo expone llamadas al sistema para **crear y controlar procesos**. (`system()`, `fork()`, `exec()`).
 - `fork()`
 - Duplica el proceso actual (crea hijo).
 - Hijo y padre continúan ejecutando en paralelo.
 - El hijo recibe copia del espacio de direcciones del padre (Copy-on-write en sistemas modernos).
 - El padre recibe como resultado el pid del proceso hijo.
 - El hijo recibe como resultado el numero 0.
 - `exec()`
 - Reemplaza el código del proceso por otro programa.
 - Mantiene mismo PID, pero nuevo binario cargado.

- Usado para ejecutar programas externos.
- `wait()`
 - Padre espera a que termine el hijo.
 - Recupera código de salida.
 - Cuando el hijo termina, el padre obtiene el código de status del hijo.

Ejecutar en la CPU

Cuando decimos que un **proceso está en ejecución (estado *Running*)**, significa que la **CPU le ha asignado tiempo de procesamiento** y está ejecutando sus instrucciones.

Componentes que intervienen

1. Contador de programa (PC / Program Counter)

- Señala la **próxima instrucción** a ejecutar.
- Cada vez que la CPU completa una instrucción, el PC avanza.

2. Registros de CPU

- Guardan valores temporales que el proceso necesita (acumuladores, índices, punteros).
- Son los datos de trabajo inmediato de la CPU.

3. Stack (pila del proceso)

- Guarda parámetros de funciones, direcciones de retorno y variables locales.
- Permite que el proceso pueda hacer llamadas a funciones y volver correctamente.

4. Datos y Heap

- Variables globales y memoria dinámica asignada (`malloc` , `new`).

Ejecución y cambios de contexto

- El **scheduler** del sistema operativo decide qué proceso entra a la CPU.
- Cuando un proceso pasa a estado *Running*:

1. Se **cargan en la CPU** los registros y PC guardados en el **PCB (Process Control Block)**.
2. El proceso ejecuta sus instrucciones.
- Si el proceso es interrumpido (fin de quantum, E/S, interrupción de hardware):
 1. Se **guarda el contexto actual** (registros, PC, estado) en el PCB.
 2. El scheduler selecciona otro proceso y se repite el ciclo.

Este mecanismo se llama **cambio de contexto (context switch)**.

Quantum y planificación

- En sistemas multitarea con **scheduling con desalojo**, cada proceso recibe un **quantum** (tiempo máximo en CPU).
- Si se acaba el quantum, se produce una **interrupción de timer** → el proceso pasa a *Ready* y otro toma la CPU.

Hacer un system call.



POSIX: Portable Operating System Interface; X: UNIX.

Un proceso también puede hacer llamadas al sistema.

Por ejemplo:

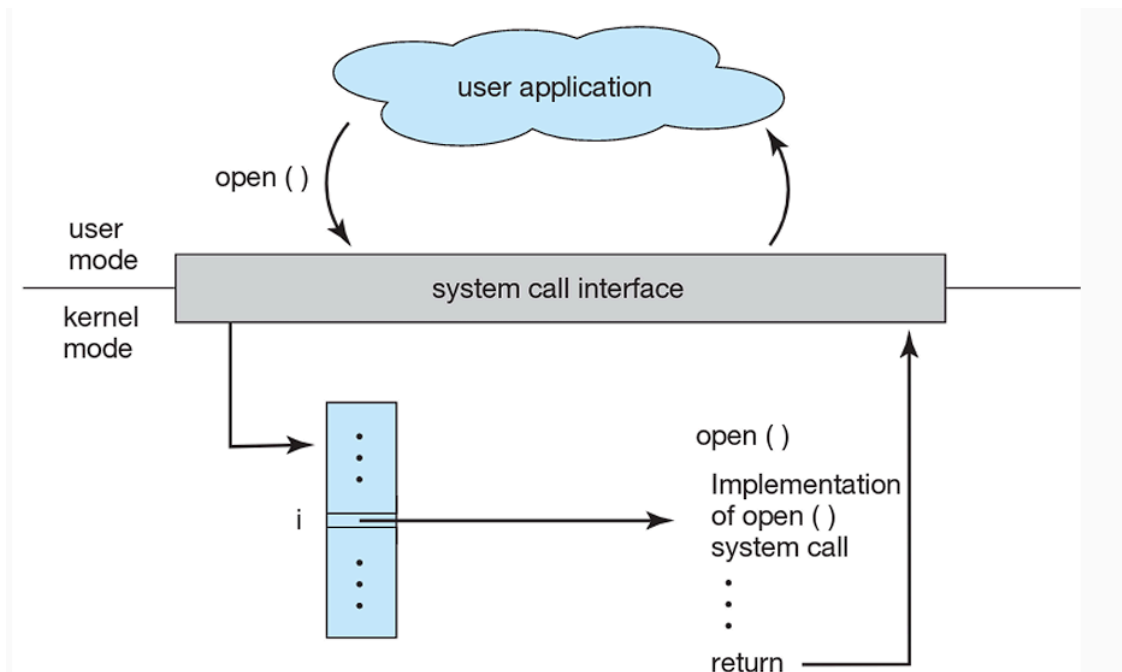
- `fork()` o `exec()`
- `write()` (imprimir en pantalla)

En todas se debe llamar al kernel, lo cual requiere cambiar de privilegio, un cambio de contexto, a veces una interrupción, todo esto toma tiempo.

Syscall

- Las syscalls proveen una interfaz a los servicios brindados por el sistema operativo: la API (Application Programming Interface) del SO.
- En general, se usa una interrupción para pasar a modo kernel y los parametros se pasan usando registros.

- Normalmente se las utiliza a través de wrapper functions en C.



Wrapper functions

- ☒ Mejora la portabilidad.
- ☒ No requiere programar en Assembler.
- ☒ Permite interactuar con el sistema con mayor sencillez

Realizar entrada/salida a los dispositivos (E/S)



La E/S es muy lenta

Quedarse bloqueado esperando es un desperdicio de tiempo, implica hacer busy waiting, es decir, gastar ciclos de CPU.

Existen mejores alternativas:

- Polling
- Interrupciones

Busy waiting

El proceso no libera la CPU. Un único proceso en ejecución a la vez.

Polling

El proceso libera la CPU, pero todavía recibe un quantum que desperdicia hasta que la E/S esté terminada.

Interrupciones

Esto permite la multiprogramación.

- El SO no le otorga más quanta al proceso hasta que su E/S esté lista.
- El HW comunica que la E/S terminó mediante una interrupción.
- La interrupción es atendida por el SO, que en ese momento "despierta al proceso".

E/S bloqueante vs. no bloqueante

- **Bloqueante** → el proceso espera a que termine la operación de E/S.
- **No bloqueante (asincrónica)** → el proceso pide E/S y sigue ejecutando; se le notifica cuando termina.

Ejemplo:

- Lectura de teclado → bloqueante.
 - Operaciones de red → muchas veces no bloqueantes (select/poll/epoll).
-

Estados de un proceso

1. **New** → creado.
2. **Ready** → listo para ejecutar, esperando CPU.
3. **Running** → en ejecución.
4. **Waiting (Blocked)** → esperando evento de E/S.
5. **Terminated** → finalizado.
 - a. El proceso indica al sistema operativo que ya puede liberar todos sus recursos (exit()).
 - b. Además, indica su status de terminación (usualmente, un código numérico).
 - c. Este código de status le es reportado al padre.

💡 El SO mantiene un **PCB (Process Control Block)** con toda la info del proceso:

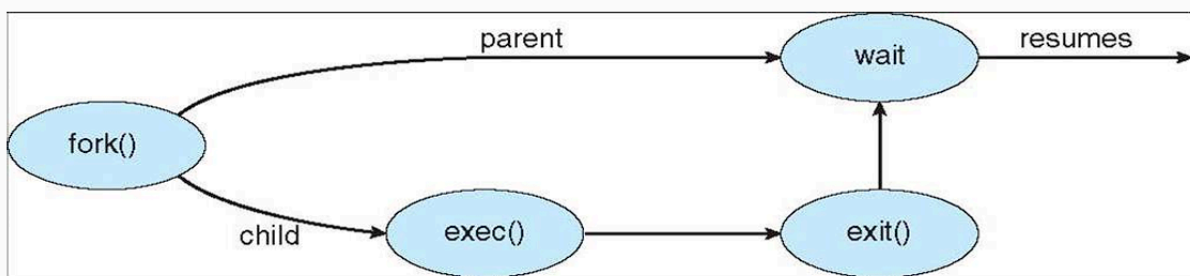
- PID, estado, registros, memoria asignada, archivos abiertos, scheduling info.

Árbol de procesos



Todos los procesos están organizados jerárquicamente, como un árbol.

Cuando el SO comienza, lanza un proceso que se suele llamar init o systemd.



Cuando lanzamos un programa desde el shell, ¿qué sucede?

- El shell hace un `fork()`.
- El hijo hace un `exec()`.

Señales y Handlers



Las **señales** son un mecanismo que incorporan los sistemas operativos POSIX, y que permite notificar a un proceso la ocurrencia de un evento.

Cuando un proceso recibe una **señal**, su ejecución normal se **interrumpe** y se transfiere el control a una rutina especial llamada **handler** (o manejador de

señal).

- Cada tipo de señal tiene asociado un **handler por defecto**, pero este puede modificarse mediante la syscall `signal()`.
- Las señales están identificadas por un **número entero** y constantes simbólicas definidas en `<signal.h>` (ejemplos: `SIGINT`, `SIGKILL`, `SIGSEGV`).
- Algunas señales son **irreemplazables**:
 - `SIGKILL` y `SIGSTOP` **no pueden ser bloqueadas ni redefinidas**.
- El envío de señales puede hacerse de dos formas:
 - Desde la terminal, con el comando `kill`.
 - Desde otro proceso, mediante la syscall `kill()`.